

# LoopDetectR: Comprehensive Feedback Loop Detection in ODE models

Katharina Baum

7/02/2020

## Contents

|                                                                                      |   |
|--------------------------------------------------------------------------------------|---|
| Installation . . . . .                                                               | 1 |
| In brief and quick start . . . . .                                                   | 1 |
| Introduction . . . . .                                                               | 2 |
| Solving the ODE model to generate variable values of interest . . . . .              | 3 |
| Calculating the Jacobian matrix . . . . .                                            | 4 |
| Computing all feedback loops and useful functions for loop search . . . . .          | 4 |
| Computing feedback loops over multiple sets of variable values of interest . . . . . | 6 |
| Comparing two loop lists . . . . .                                                   | 7 |
| References . . . . .                                                                 | 9 |

## Installation

LoopDetectR is on CRAN and can be installed within R by

```
# Download and install
install.packages("LoopDetectR")
# Load package
library("LoopDetectR")
```

Alternatively, download the package folder from [gitlab.com/kabaum/LoopDetectR](https://gitlab.com/kabaum/LoopDetectR), unzip; start an R session, and navigate to the unzipped folder and set it as working directory. Call the following R commands to install it from the local source.

```
# Install from unzipped folder (or enter the folder path as first argument)
install.packages("~/Desktop/LoopDetectR", repos=NULL, type="source")
# Load package
library("LoopDetectR")
```

## In brief and quick start

The package LoopDetectR enables determining all feedback loops of an ordinary differential equation (ODE) system at user-defined values of the model parameters and of the modelled variables.

The following call reports (up to 10) feedback loops for an ODE system determined by a function, here the example function `func_POSm4`, at variable values `s_star` (here, these are all equal to 1). Additional arguments to the example function are supplied.

```
# Load example ODE system with function func_POSm4, 4 variables
data("func_POSm4")
# Example variable values
s_star <- rep(1,4)
```

```

# Further arguments of func_POSm4, in addition: time t as argument
klin <- rep(1,8)
knonlin <- c(2.5,3)

# compute loops
res_tab <- find_loops_vset(func_POSm4,vset=list(s_star),t=1,klin=klin,
                           knonlin=knonlin,max_num_loops=10)

# The loop list is reported
res_tab$loop_rep[[1]]

##          loop length sign
## 1          1, 1      1  -1
## 2          2, 2      1  -1
## 3          3, 3      1  -1
## 4          4, 4      1  -1
## 5 3, 4, 1,...      4  -1
## 6    3, 4, 2, 3      3   1

# This is the sixth loop of the list. It is a positive feedback loop (sign in
# the loop list equals +1) of length 3 in that variable 3 regulates variable 4,
# variable 4 regulates variable 2, and variable 2 regulates variable 3.
res_tab$loop_rep[[1]][6,]

##          loop length sign
## 6 3, 4, 2, 3      3   1

# The corresponding signed Jacobian matrix
res_tab$jac_rep[[1]]

##      [,1] [,2] [,3] [,4]
## [1,]  -1   0   0  -1
## [2,]   1  -1   0   1
## [3,]   0   1  -1   0
## [4,]   0   0   1  -1

```

Here, the sixth loop in the computed loop list

## Introduction

Ordinary differential equation (ODE) models are used frequently to mathematically represent biological systems. Feedback loops are important regulatory features of biological systems and can give rise to different dynamic behavior such as multistability for positive feedback loops or oscillations for negative feedback loops.

The feedback loops in an ODE system can be detected with the help of its Jacobian matrix, the matrix of partial derivatives of the variables. It captures all interactions between the variables and gives rise to the interaction graph of the ODE model. In this graph, each modelled variable is a node and non-zero entries in the Jacobian matrix are (weighted) edges of the graph. Interactions can be positive or negative, according to the sign of the Jacobian matrix entry.

Directed path detection in this graph is used to determine all feedback loops (in graphs also called cycles or circuits) of the system. They are marked by a set of directed interactions forming a chain in which only the first and the last node (variable) is the same. Thereby, self-loops (loops of length one) can also occur.

LoopDetectR allows for detection of all loops of the graph and also reports the sign of each loop, i.e. whether it is a positive feedback loop (the number of negative interactions is even) or a negative feedback loop (the number of negative interactions is uneven). The output is a table that captures the order of the variables forming the loop, their length and the sign of each loop.

Jacobian determination in LoopDetectR relies on the package `numDeriv`, and path finding in graphs uses algorithms supplied in the package `igraph`.

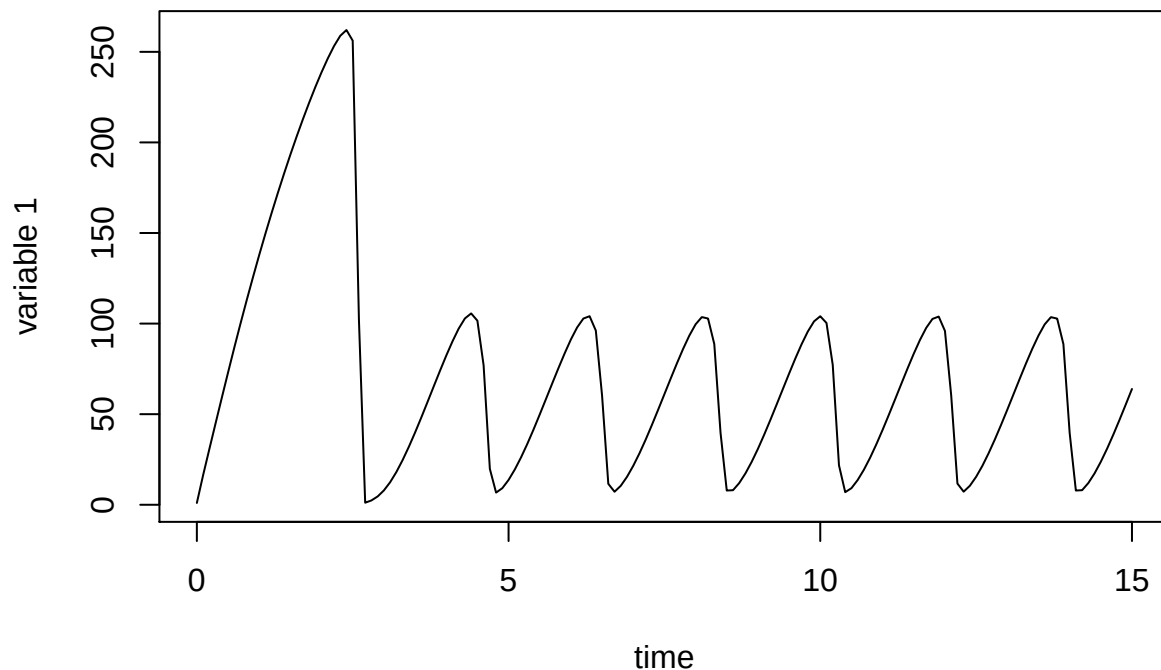
## Solving the ODE model to generate variable values of interest

Solving an ODE model can be performed with the package `deSolve`. Note: You can skip this step if you already have a point of interest in state space, or if you want to use dummy values for the variables such as `s_tar <- 1:4`.

```
# install and load package deSolve for solving the ODE
install.packages("deSolve",repos="https://cloud.r-project.org")

##
## The downloaded binary packages are in
## /var/folders/rh/4c9gc49n28bdyr6_7l10d9740000gn/T//RtmpMCiqwk/downloaded_packages
library('deSolve')

# Load example ODE system with function func_POSm4,
# Positive feedback chain model from [Baum et al., 2016], 4 variables
data(func_POSm4)
# The function func_POSm4 returns a vector, but deSolve needs the vector within
# a list as output. Therefore, we define a function that simply puts the output
# of func_POSm4 into a list:
func_POSm4_list <- function(t,x,klin,knonlin){list(func_POSm4(t,x,klin,knonlin))}
# Kinetic parameters of the model, supplied as arguments to func_POSm4
klin <- c(165,0.044,0.27,550,5000,78,4.4,5.1)
knonlin <- c(0.3,2)
# Solve the system using deSolve
sol <- deSolve::ode(y = rep(1,4), times = seq(0,15,0.1), func = func_POSm4_list,
                    parms=klin, knonlin=knonlin)
# The solution of the 4-variable system is oscillatory, showing only the first
# variable here
plot(sol[,1],sol[,2],type='l',xlab='time',ylab='variable 1')
```



```
# Set the last point of the numeric solution as point of interest, omit the
# first column (it contains the time)
s_star <- tail(sol[,2:dim(sol)[2]],n=1)
```

State variable values of interest could be steady state values, values at a specific point in time (e.g. after a stimulus) or even a set of values (see section *Determining loops over a set of variable values*).

## Calculating the Jacobian matrix

The function `jacobian` from the `numDeriv` package can be used to determine numerically the Jacobian matrix of an ODE system at a certain set of values for the variables, `s_star`. The approach is that of finite differences (with real step) or complex step approach, the latter of which is supposed to deliver more exact results [Martins et al., 2003].

The input function, in the example `func_POSm4` (positive feedback chain model from [Baum et al., 2016]) defines the time derivatives of the modelled variables as a vector:  $f_i(s) = dS_i/dt$ . Note that only those input arguments to the function that encode the modelled variables (and hence in whose direction the partial derivatives are taken) are allowed to be called `x`.

```
klin <- c(165,0.044,0.27,550,5000,78,4.4,5.1)
knonlin <- c(0.3,2)
j_matrix <- numDeriv::jacobian(func_POSm4,s_star,method="complex",
                              t=1,klin=klin,knonlin=knonlin,)
j_matrix
```

```
##           [,1] [,2] [,3] [,4]
## [1,] -1.2396132  0  0.0 -85.9719
## [2,]  0.9696132 -5550  0.0  85.9719
## [3,]  0.0000000  550 -82.4  0.0000
## [4,]  0.0000000  0  78.0 -5.1000
```

```
signed_jacobian <- sign(j_matrix)
```

The (i,j)th entry of the Jacobian matrix denotes the partial derivative of variable  $S_i$  with respect to variable  $S_j$ ,  $J_{ij} = \delta S_i / \delta S_j$ , which is positive if  $S_j$  has a direct positive effect on  $S_i$ , negative if  $S_j$  has a direct negative effect on  $S_i$  and zero if  $S_j$  does not have a direct effect on  $S_i$ . For example, the entry in row 2, column 4,  $J_{24}$ , of the `signed_jacobian` matrix above is positive, meaning that in the underlying ODE model, variable 4 positively regulates variable 1.

## Computing all feedback loops and useful functions for loop search

The Jacobian matrix is used to compute feedback loops in the generated interaction graph. The default function for this is `find_loops`, in that strongly connected components are determined to reduce runtime. For smaller systems, the function `find_loops_noscc` skips this step and thus can be faster. The optional second input argument, `max_num_loops`, sets an upper limit to the number of detected and reported loops and thus can prevent overly long runtime (but also potentially not all loops are returned).

```
# Determine the loop_list from Jacobian matrix j_matrix
loop_list <- find_loops(j_matrix)
loop_list
```

```
##           loop length sign
## 1           1, 1      1   -1
## 2           2, 2      1   -1
## 3           3, 3      1   -1
## 4           4, 4      1   -1
## 5 3, 4, 1,...      4   -1
```

```
## 6    3, 4, 2, 3      3      1
# The signed Jacobian matrix can be supplied instead, delivering the same results
loop_list <- find_loops(signed_jacobian)
loop_list

##           loop length sign
## 1          1, 1        1  -1
## 2          2, 2        1  -1
## 3          3, 3        1  -1
## 4          4, 4        1  -1
## 5 3, 4, 1,....      4  -1
## 6    3, 4, 2, 3      3      1
```

The output loop list is an R data.frame with one row for each detected loop. The column `loop` contains the order of the variables that form the loop (as vector in a list); `length` contains the loop length (i.e. the number of variables involved); `sign` denotes whether the loop is negative, -1, or positive, 1. The data.frame can be queried as usual for single loops or loops of a certain length or sign.

```
# Retrieve fifth loop
loop_list[5,1]

## [[1]]
## [1] 3 4 1 2 3

# In order to obtain the vector of the order of variables of a loop, you have to
# call the list element
loop_list[5,1][[1]]
```

```
## [1] 3 4 1 2 3

# Retrieve all loops of length 4
loop_list[loop_list$length==4,]
```

```
##           loop length sign
## 5 3, 4, 1,....      4  -1
```

The LoopDetectR function `loop_summary` provides a convenient report on total number of loops, subdivided by their lengths (`len_i`) and signs (`all`, `pos`, `neg`).

```
loop_summary(loop_list)

##      len_1 len_2 len_3 len_4
## all      4     0     1     1
## pos      0     0     1     0
## neg      4     0     0     1
```

One can filter the loop list for loops containing specific variables, for example the one with index 2:

```
# Index of node of interest
noi <- 2
# Return all loops from loop_list containing node 2
loop_list[vapply(loop_list$loop,function(x){noi %in% x},logical(1)),]

##           loop length sign
## 2          2, 2        1  -1
## 5 3, 4, 1,....      4  -1
## 6    3, 4, 2, 3      3      1
```

The LoopDetectR function `find_edge` can be used to search a loop list for loops containing specific edges defined by the indices of the ingoing and outgoing nodes. This example returns the indices of all loops with a

regulation of node 3 by node 2. These are only two here.

```
# Obtain the indices of the loops with edge '2 regulates 3' in loop_list
loop_edge_ind <- find_edge(loop_list,source_node=2,target_node=3);
loops_with_edge_2_to_3 <- loop_list[loop_edge_ind,]
loops_with_edge_2_to_3
```

```
##           loop length sign
## 5 3, 4, 1,...      4   -1
## 6   3, 4, 2, 3      3    1
```

Loop lists can be saved and loaded as R objects using the usual save (or saveRDS) and load (loadRDS) R functions. Saving the loop list into table files is possible, but the vectors containing the loops are saved as string "c(x,y,z)" and need special handling when loading back into R or into other programming language frameworks.

```
# This writes a file 'loopleft_func_POSm4.RData'
save(loop_list,file='loopleft_func_POSm4.RData')
# This loads the object 'loop_list' into the workspace
load('loopleft_func_POSm4.RData')

# This saves the loop list into a table with separation by tab
write.table(loop_list,file='loopleft_func_POSm4.txt',sep='\t',quote=F,row.names=F)
# This reads the loop list back into the R session, as object ll
ll <- read.table('loopleft_func_POSm4.txt',header=T,sep='\t')
ll
```

```
##           loop length sign
## 1      c(1, 1)          1   -1
## 2      c(2, 2)          1   -1
## 3      c(3, 3)          1   -1
## 4      c(4, 4)          1   -1
## 5 c(3, 4, 1, 2, 3)      4   -1
## 6      c(3, 4, 2, 3)      3    1
```

```
# This transforms the "c(x,y,z)"-entries into the format used by LoopDetectR
ll$loop <- lapply(ll$loop,function(x){eval(parse(text=x))})
ll
```

```
##           loop length sign
## 1      1, 1            1   -1
## 2      2, 2            1   -1
## 3      3, 3            1   -1
## 4      4, 4            1   -1
## 5 3, 4, 1, 2, 3        4   -1
## 6   3, 4, 2, 3         3    1
```

## Computing feedback loops over multiple sets of variable values of interest

In this example of a model of the bacterial cell cycle [Li et al., 2008], it is demonstrated how feedback loops can be determined over multiple sets of variable values. Here, it is focused on the solution of the ODE systems along the time axis (provided as data in the package, *li08\_solution.RData*).

```
# Load in the example ODE model function func_li08, the kinetic parameters are
# defined within the function, and the function returns a vector of the time
# derivatives (in the same order as the modelled variables in the arguments)
data("func_li08")
# Load sets of variable values (solution to the ODE over time)
```

```

data("li08_solution") #loads the data.frame li08_solution, columns: variables
# Cast the solution into the correct list format and remove time (first column)
li08_sol_list <- as.list(as.data.frame(t(li08_solution[,-1])))
# Compute all different loop lists along the solution
res_tab <- find_loops_vset(func_li08,vset=li08_sol_list,t=1,
                          compute_full_list=FALSE)

```

The solutions of the example ODE model give rise to seven different loop lists that are saved as elements of the list `res_tab$loop_rep`. Here, two examples of resulting loop lists are given (without self-loops).

```

loop_list_2 <- res_tab$loop_rep[[2]][res_tab$loop_rep[[2]]$length>1,]
loop_list_2

```

```

##           loop length sign
## 16 6, 7, 1, 6         3   -1
## 17   6, 7, 6         2    1
## 18   2, 3, 2         2   -1

```

```

loop_list_7 <- res_tab$loop_rep[[7]][res_tab$loop_rep[[7]]$length>1,]
loop_list_7

```

```

##           loop length sign
## 15      1, 2, 1         2   -1
## 16     1, 3, 2, 1        3    1
## 17      2, 3, 2         2   -1
## 18 5, 7, 1, ....        4   -1
## 19 6, 7, 1, ....        5    1
## 20     6, 7, 1, 6        3   -1
## 21      6, 7, 6         2    1
## 22 10, 11, ....         6    1
## 23 10, 12, ....         3   -1
## 24 10, 13, ....         4   -1
## 25 10, 14, ....         8   -1
## 26 10, 14, ....         7    1

```

The entry `res_tab$loop_rep_index` is a vector of the same length as the number of different states at which the loops were determined (`length(li08_sol_list)`), and returns which entry of `res_tab$loop_rep` belongs to each input state.

```

# Determine the loop list belonging to the 10th up to 20th input state.
# It is loop_list_2 for all of these input states.
res_tab$loop_rep_index[10:20]

```

```

## [1] 2 2 2 2 2 2 2 2 2 2 2

```

Similarly, `res_tab$jac_rep` and `res_tab$jac_rep_index` capture the different Jacobian matrices and for each input state which Jacobian belongs to it.

Results of this analysis could be plotted along the solution and analyzed further to discover reasons of changing loops. Please note that in order to obtain the sample solution in *li08\_solution.RData* also event functions are required; the solution cannot be retrieved from integrating `func_li08` alone. Please refer to the model's publication [Li et al., 2008] for details.

## Comparing two loop lists

LoopDetectR provides a function for comparing the loops of two systems, `compare_loop_list`. For a meaningful comparison, the loop indices in the compared systems should point to the same variables. This could be the case when regulations change within one system between different sets of variables of interest

(along a dynamic trajectory, at different steady states of the system), or when comparing different systems in which one or more regulations are altered (as in the below example the positive feedback chain model vs. the negative feedback chain model, [Baum et al., 2016]).

```
# Load the ODE function of the positive feedback chain model
data(func_POSm4)
# Set kinetic parameters
klin <- c(165,0.044,0.27,550,5000,78,4.4,5.1)
knonlin <- c(0.3,2)
# Compute the Jacobian matrix of the system at the state [1,1,1,1]
j_matrix <- numDeriv::jacobian(func_POSm4,rep(1,4),method="complex",
                                t=1, klin=klin, knonlin=knonlin)
# Compute all loops for this Jacobian
loop_list_pos <- find_loops(j_matrix)

# Function with negative regulation. The altered regulation affects
# two entries of the Jacobian matrix. Parameter values and the set of
# variable values remain identical.
j_matrix_neg <- sign(j_matrix)
j_matrix_neg[1:2,4] <- -sign(j_matrix[1:2,4])
# Compute the loop list for this Jacobian with altered regulation
loop_list_neg <- find_loops(j_matrix_neg);

# Compute comparison
comp_inds <- compare_loop_list(loop_list_pos,loop_list_neg)
# Only the four self-loops remain identical in both systems (saved in ind_a_id).
loop_list_pos[comp_inds$ind_a_id,]

##      loop length sign
## 1 1, 1          1  -1
## 2 2, 2          1  -1
## 3 3, 3          1  -1
## 4 4, 4          1  -1

# ind_b_id saves the indices of the corresponding loops in the negatively
# regulated system.
loop_list_neg[comp_inds$ind_b_id,]

##      loop length sign
## 1 1, 1          1  -1
## 2 2, 2          1  -1
## 3 3, 3          1  -1
## 4 4, 4          1  -1

# Two loops are the same in both systems but they have switched their signs.
# Their indices in the first list are saved in ind_a_switch
loop_list_pos[comp_inds$ind_a_switch,]

##      loop length sign
## 5 3, 4, 1,...    4  -1
## 6  3, 4, 2, 3    3   1

# Their indices in the second list are saved in ind_b_switch
loop_list_neg[comp_inds$ind_b_switch,]

##      loop length sign
## 5 3, 4, 1,...    4   1
```

```
## 6 3, 4, 2, 3 3 -1
```

```
# All loops in the positively regulated system do also occur in the negatively  
# regulated system, i.e. ind_a_notin is empty.  
comp_inds$ind_a_notin
```

```
## integer(0)
```

## References

Baum K, Politi AZ, Kofahl B, Steuer R, Wolf J. Feedback, Mass Conservation and Reaction Kinetics Impact the Robustness of Cellular Oscillations. PLoS Comput Biol. 2016;12(12):e1005298.

Li S, Brazhnik P, Sobral B, Tyson JJ. A Quantitative Study of the Division Cycle of Caulobacter crescentus Stalked Cells. Plos Comput Biol. 2008;4(1):e9.

Martins JRRA, Sturdza P, Alonso JJ. The complex-step derivative approximation. ACM Trans Math Softw. 2003;29(3):245–62.